

成绩

--

华中科技大学

课程设计报告

课 程： 操作系统设计与实践

课设名称： 阻塞和非阻塞式设备驱动开发和
内核缓冲区读写同步机制设计

院 系： 软件学院

专业班级： 软件 2305 班

学 号： U202317XXX

姓 名： xxx

2025 年 06 月 13 日

目录

课程设计报告	1
1 课设目的.....	1
2 课程设计内容.....	1
3 程序设计思路.....	2
1. 循环缓冲区字符设备驱动编写	2
1.1 整体分析与设计.....	2
1.2 程序依赖的库函数.....	3
1.3 核心模块设计与划分	4
1.4 核心数据结构.....	5
1.5 各模块详解.....	6
1.6 操作流程.....	16
2. 测试程序编写	17
2.1 整体分析与设计.....	17
2.2 buffer_writer.c / buffer_reader.c - 缓冲区写入/读取测试	17
2.3 buffer_nonblock.c - 缓冲区非阻塞读写测试	18
2.4 buffer_monitor.c - 缓冲区与进程状态监控程序.....	18
4 实验程序的难点或核心技术分析	19
1. 核心思路概述.....	19
2. 关键机制.....	19
2.1 kfifo 循环缓冲区	19
2.2 阻塞读写实现.....	20
2.3 测试程序设计.....	21
5 开发和运行环境的配置	21
1. 开发环境.....	21
2. Makefile 编写	22
6 运行和测试过程.....	24
1. 编译和加载内核模块和测试程序	24
2. 运行测试程序	24
3. 卸载驱动程序并清理.....	26
7 实验心得和建议.....	26
8 学习和编程实现参考网址	27

1 课设目的

1. 理解和应用“设备就是文件”的概念，实际应用中应用程序就像读写普通文件一样可以标准系统调用来操作硬件，极大地简化了应用编程
2. 熟悉 Linux 设备驱动程序开发过程，熟悉 Linux 设备驱动程序的开发流程，包括模块的加载与卸载、设备号的申请与释放、file_operations 结构体的填充以及字符设备的注册与注销
3. 理解设备的阻塞和非阻塞工作机制
4. 理解和应用内核同步机制，学习并使用“等待队列”（wait queue）来精确地管理因设备资源（如缓冲区）不可用而需要休眠的进程，并在资源可用时唤醒它们，从而实现高效的进程调度和 CPU 资源利用。
5. 编程技能：

内核编程：学会在内核空间编写代码，遵循内核编码规范，并了解如何编译、加载和卸载内核模块

驱动开发：掌握字符设备驱动的基本框架

同步机制：使用等待队列等同步原语，来处理驱动中的并发访问问题，实现高效的阻塞 I/O

proc 接口文件系统：用户可以在不通过驱动程序 read/write 的情况下方便地查看设备的状态和内部信息

2 课程设计内容

1. 编写一个设备驱动程序（本实验实现的是字符设备）

-
- 1) 内设固定大小缓冲区 BUFFER, 读/写不遗漏不重复 (读指针 pRead, 写指针 pWrite), 当缓冲区塞满时再 write 会阻塞应用程序直到别的应用程序读走一些数据, 然后再唤醒阻塞的 write 进程;
 - 2) 实现设备的阻塞和非阻塞两种工作方式;
 - 3) 实现 proc 接口, 把驱动工作情况记录在 proc 文件中;
2. 编写若干具有读/写功能的测试程序
 - 1) 观察: 缓冲区变化(proc 文件), 读/写进程阻塞/唤醒的情况
 - 2) ps 命令查看进程状态

3 程序设计思路

1. 循环缓冲区字符设备驱动编写

1.1 整体分析与设计

本程序旨在实现一个功能相对完善的字符设备驱动, 并创立了一个固定大小的循环缓冲区。用户空间的应用程序可以通过标准的设备文件接口 (如 open, read, write, close) 与该设备进行交互。核心设计目标如下:

- 数据缓冲: 驱动内部维护一个缓冲区, 应用程序写入的数据先存入缓冲区, 读取时从缓冲区取出。
- 先进先出 (FIFO): 保证数据的读写顺序。
- 阻塞/非阻塞 IO:
 - 阻塞模式: 当缓冲区满时, 写操作会阻塞, 直到有空间可用; 当缓冲区空时, 读操作会阻塞, 直到有数据可读。

○ 非阻塞模式：当无法立即完成操作时（如缓冲区满/空），操作立即返回错误码 (-EAGAIN)。

- 并发控制：允许多个进程同时访问设备，通过锁机制保证数据一致性。
- 状态监控：通过 `proc` 文件系统接口，向上层应用暴露驱动内部状态，如缓冲区使用情况、总读写字节数等。

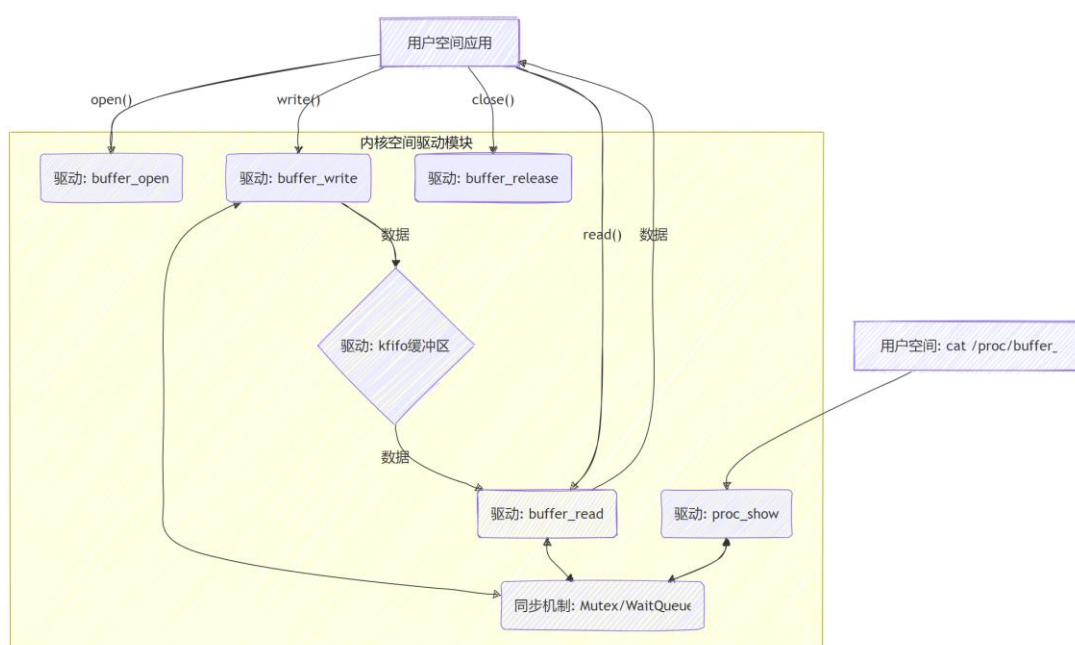


图 1 驱动程序整体结构设计

1.2 程序依赖的库函数

- `linux/module.h`: 内核模块必需，包含模块加载/卸载宏 (`module_init`, `module_exit`) 和模块信息宏 (`MODULE_LICENSE` 等)。
- `linux/kernel.h`: 内核基本定义，如 `KERN_INFO` 等日志级别。
- `linux/fs.h`: 文件系统相关定义，如 `struct file_operations`, `struct file`, `struct inode` 等。
- `linux/cdev.h`: 字符设备相关定义，如 `struct cdev`, `cdev_init`, `cdev_add`。
- `linux/device.h`: 设备模型相关，用于创建设备文件，如 `class_create`,

`device_create`。

- `linux/slab.h`: 内核内存分配函数, 如 `kmalloc`, `kfree`。
- `linux/wait.h`: 等待队列相关, 用于实现阻塞操作, 如 `wait_queue_head_t`, `wait_event_interruptible`, `wake_up_interruptible`。
- `linux/mutex.h`: 互斥锁相关, 用于并发控制, 如 `struct mutex`, `mutex_init`, `mutex_lock`, `mutex_unlock`。
- `linux/proc_fs.h`: `proc` 文件系统相关, 如 `proc_create_data`, `proc_remove`。
- `linux/seq_file.h`: 用于 `proc` 文件系统输出序列化数据, 简化 `proc` 文件内容的生成, 如 `struct seq_file`, `single_open`, `seq_printf`。
- `linux/kfifo.h`: 内核 FIFO (先进先出) 缓冲区实现, 如 `struct kfifo`, `kfifo_alloc`, `kfifo_put` 等。

1.3 核心模块设计与划分

驱动程序可划分为以下几个主要模块:

- 内核模块管理:
 - 功能: 负责驱动程序的加载和卸载, 是所有内核模块的标准入口和出口。
 - 涉及代码: `module_init(buffer_init)`, `module_exit(buffer_exit)`, `MODULE_LICENSE`, `MODULE_AUTHOR` 等宏。
- 字符设备管理:
 - 功能: 负责字符设备的注册、创建和注销, 包括设备号的申请与释放、设备类的创建与销毁、`cdev` 结构的初始化与添加/删除以及在 `/dev` 目录下创建设备节点。

-
- 涉及代码: `alloc_chrdev_region`, `unregister_chrdev_region`, `class_create`, `class_destroy`, `cdev_init`, `cdev_add`, `cdev_del`, `device_create`, `device_destroy`, 以及文件操作中的 `buffer_open` 和 `buffer_release`。

- 核心数据处理与 I/O 操作

- 功能: 驱动的核心功能模块, 负责管理循环缓冲区 (kfifo), 处理用户空间的读写请求, 实现阻塞和非阻塞 I/O 机制, 并确保数据访问的同步。
- 涉及函数及数据结构: `struct buffer_dev`, `kfifo_alloc`, `kfifo_free`, `buffer_read` (包含数据拷贝、阻塞/非阻塞逻辑、唤醒写者), `buffer_write` (包含数据拷贝、阻塞/非阻塞逻辑、唤醒读者), `mutex_init`, `mutex_lock_interruptible`, `mutex_unlock`, `init_waitqueue_head`, `wait_event_interruptible`, `wake_up_interruptible`

- Proc 文件系统接口

- 功能: 提供一个 `/proc` 文件系统接口, 允许用户空间通过读取该 `proc` 文件来查看缓冲区的当前状态 (如容量、可用数据、空闲空间、读写字节数等)。
- 涉及函数及数据结构: `proc_create_data`, `proc_remove`, `buffer_proc_fops` (及其成员 `buffer_proc_open`, `buffer_proc_show`), `seq_file` API。

1.4 核心数据结构

- 设备私有数据结构(`buffer_dev`), 定义设备的核心数据:

- `fifo`: `kfifo` 结构体, 用于实现循环缓冲区。
- `lock`: 互斥锁, 用于保护对 `buffer_dev` 结构中共享资源的并发访问 (主要

是 `fifo`、`bytes_read`、`bytes_written`)。

- `read_queue`: 等待队列头, 用于在缓冲区为空时阻塞读进程。
- `write_queue`: 等待队列头, 用于在缓冲区为满时阻塞写进程。
- `cdev`: 内嵌的字符设备结构体, 用于向内核注册该字符设备。
- `bytes_read`: 无符号整型, 累计从设备读取的总字节数。
- `bytes_written`: 无符号整型, 累计向设备写入的总字节数。
- 文件操作接口 (`buffer_fops`): 实现标准的字符设备文件操作, 如 `open`, `release`, `read`, `write`。
- Proc 文件系统接口 (`buffer_proc_fops`): 允许用户通过读取 `/proc/buffer_status` 文件来查看缓冲区的当前状态, 如容量、可用数据、剩余空间等。

1.5 各模块详解

1.5.1 内核模块管理

- 核心概念:
 - 内核模块 (Kernel Module): 允许在内核运行时动态加载和卸载代码的功能单元。驱动程序通常以此形式存在。
 - `module_init(init_function)`: 宏, 指定模块加载时调用的初始化函数。
 - `module_exit(exit_function)`: 宏, 指定模块卸载时调用的清理函数。
 - `MODULE_LICENSE("GPL")`: 声明模块的许可证, 影响内核符号的可见性等。
 - `MODULE_AUTHOR`, `MODULE_DESCRIPTION`, `MODULE_VERSION`: 提供模块的元数据。

- 原理:

- insmod 命令加载模块时, 内核执行 `init_function`。
- rmmod 命令卸载模块时, 内核执行 `exit_function`。
- 这些函数负责申请和释放模块运行所需的资源。

- 设计思路:

- `buffer_init()`: 负责申请所有必要的资源 (设备号、内存、初始化锁和队列等)、注册字符设备、创建设备类和设备文件节点、创建 `proc` 文件;
- `buffer_exit()`: 作为模块的出口, 执行所有必要的清理步骤 (资源释放、设备注销), 即以与初始化相反的顺序释放所有资源, 确保模块卸载后系统状态的干净。

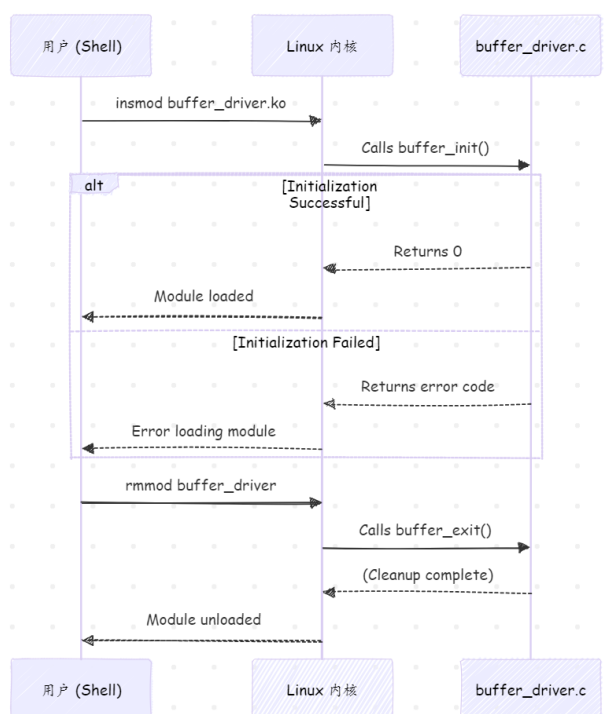


图 2 模块的初始化与卸载

1.5.2 字符设备管理

- 核心概念:

-
- 字符设备 (Character Device): 按字节流进行数据传输的设备。
 - 设备号 (dev_t): 内核中唯一标识设备的号码, 由主设备号 (Major) 和次设备号 (Minor) 组成。
 - alloc_chrdev_region() / unregister_chrdev_region(): 动态分配/释放设备号。
 - struct cdev: 内核中代表字符设备的结构, 核心是 struct file_operations *ops, 指向设备的操作函数集。
 - cdev_init() / cdev_add() / cdev_del(): 初始化、添加 (注册到内核)、删除 cdev 结构。
 - 设备类 (struct class): 用于在 sysfs 中组织设备, 并配合 udevd (或 mdev) 自动创建设备节点。
 - class_create() / class_destroy(): 创建/销毁设备类。
 - 设备节点 (/dev/DEVICE_NAME): 用户空间访问设备的入口文件。
 - device_create() / device_destroy(): 在指定类下创建/销毁设备节点。
 - struct file_operations (buffer_fops): 定义了设备支持的操作 (open, release, read, write, poll 等), 在上面已经提及。
 - struct file (filp): 代表一个打开的文件实例, filp->private_data 常用于存储设备特定的数据。
 - struct inode: 代表一个文件 (元数据), inode->i_cdev 指向字符设备的 cdev 结构。
 - container_of(ptr, type, member): 从结构体成员指针获取其父结构体的指针。

● 原理:

-
- 分配设备号: 为设备获取一个唯一的标识。
 - 初始化 cdev: 将 file_operations 与 cdev 关联。
 - 添加 cdev: 使内核能够通过设备号找到驱动。
 - 创建设备类和设备节点: 在 /dev 目录下创建用户可见的设备文件, 方便访问。
 - Open/Release: open 时, 通过 inode 获取设备结构体并存入 filp->private_data; release 时进行相应清理。
 - 设计思路:
 - buffer_init():
 1. 分配主设备号。
 2. 创建设备类 (buffer_class)。
 3. 分配并初始化自定义的设备结构体 buffer_device (类型为 struct buffer_dev), 它将包含 cdev 成员。
 4. 用 buffer_fops 初始化 buffer_device->cdev。
 5. 将 buffer_device->cdev 添加到内核。
 6. 使用 device_create() 创建设备文件 /dev/buffer_dev。
 - buffer_exit(): 执行与初始化相反的清理操作。
 - buffer_open(): 当用户打开 /dev/buffer_dev 时, 通过 inode->i_cdev 找到 buffer_device, 并将其赋给 filp->private_data, 以便后续 read/write/poll 操作能方便地访问设备数据。
 - buffer_release(): 在此简单驱动中无特殊操作。

1.5.3 核心数据处理与 I/O 操作

- 核心概念:

- struct buffer_dev (设备私有数据), 上面已经提及

- kfifo 相关 API:

- kfifo_alloc(fifo, size, gfp_mask): 分配并初始化 kfifo。

- kfifo_free(fifo): 释放 kfifo。

- kfifo_len(fifo): kfifo 中已用数据长度。

- kfifo_avail(fifo): kfifo 中可用空间长度。

- kfifo_is_empty(fifo) / kfifo_is_full(fifo): 判断空/满。

- kfifo_to_user(fifo, to, len, copied_len_ptr): 从 kfifo 拷贝数据到用户空间 to, 最多 len 字节, 实际拷贝长度存入 copied_len_ptr。内部处理 copy_to_user。

- kfifo_from_user(fifo, from, len, copied_len_ptr): 从用户空间 from 拷贝数据到 kfifo。内部处理 copy_from_user。

- 互斥锁 (Mutex): mutex_init(), mutex_lock_interruptible(), mutex_unlock()。
确保临界区代码的原子性。

- 等待队列 (Wait Queue): init_waitqueue_head(),
wait_event_interruptible(queue, condition), wake_up_interruptible(queue)。实现进程的条件睡眠和唤醒。

- 阻塞 I/O: 若操作不能立即完成 (读空/写满), 进程睡眠直到条件满足。

- 非阻塞 I/O (O_NONBLOCK): 若操作不能立即完成, 立即返回 -EAGAIN。

- 原理:

- 数据存储: `kfifo` 作为核心的循环缓冲区。

- 同步: `mutex` 保护对 `kfifo` 和统计变量的访问, 以及对等待队列条件的检查。

- 阻塞读 (`buffer_read`):

- 1. 获取锁。

- 2. 循环检查 `kfifo_is_empty()`:

- 如果为空:

- 非阻塞: 释放锁, 返回 `-EAGAIN`。

- 阻塞: 释放锁, 调用 `wait_event_interruptible(dev->read_queue, !kfifo_is_empty(&dev->fifo))` 使进程睡眠。被唤醒后, 必须重新获取锁并再次检查条件 (因为可能是假唤醒或多个读者被唤醒)。

- 如果非空: 跳出循环。

- 3. 计算可读数据量 `len_to_copy` (用户请求 `count` 和 `kfifo_len()` 的最小值)。

- 4. `kfifo_to_user()` 拷贝数据到用户缓冲区。

- 5. 更新 `dev->bytes_read`。

- 6. 唤醒可能在 `write_queue` 上等待的写者 (`wake_up_interruptible(&dev->write_queue)`)。

- 7. 释放锁。返回实际读取的字节数。

- 阻塞写 (`buffer_write`):

-
1. 获取锁。
 2. 循环检查 `kfifo_avail() == 0` (或 `kfifo_is_full()`):
 - 如果已满:
 - 非阻塞: 释放锁, 返回 `-EAGAIN`。
 - 阻塞: 释放锁, 调用 `wait_event_interruptible(dev->write_queue, kfifo_avail(&dev->fifo) > 0)` 使进程睡眠。被唤醒后, 重新获取锁并再次检查条件。
 - 如果未满: 跳出循环。
 3. 计算可写数据量 `len_to_copy` (用户请求 `count` 和 `kfifo_avail()` 的最小值)。
 4. `kfifo_from_user()` 从用户缓冲区拷贝数据到 `kfifo`。
 5. 更新 `dev->bytes_written`。
 6. 唤醒可能在 `read_queue` 上等待的读者 (`wake_up_interruptible(&dev->read_queue)`)。
 7. 释放锁。返回实际写入的字节数。
 - 错误处理: `mutex_lock_interruptible` 和 `wait_event_interruptible` 可能因信号中断返回 `-ERESTARTSYS`。`kfifo_to/from_user` 可能因 `copy_to/from_user` 失败而返回错误。
 - 设计思路
 - `buffer_read()` 和 `buffer_write()` 严格遵循 "加锁 -> 检查条件 -> (若不满足且阻塞则释放锁、睡眠、唤醒后重加锁、重检查) -> 操作 -> 唤

醒对方 -> 解锁" 的模式。

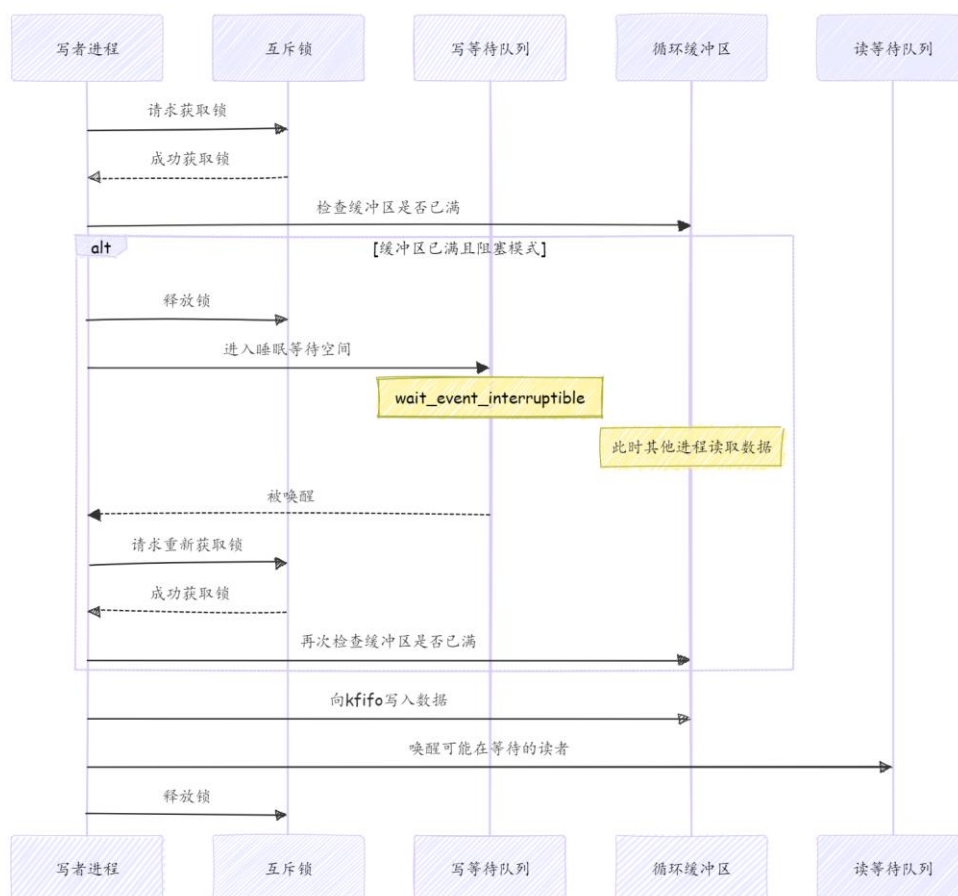


图 3 示例：阻塞写中的锁与等待队列交互

1.5.4 Proc 文件系统接口

● 核心概念:

- Proc 文件系统 (/proc): 内核与用户空间通信的虚拟文件系统，用于显示内核状态和数据。
- `proc_create_data(name, mode, parent, proc_ops, data)`: 创建一个 /proc 文件。
 - `name`: 文件名 (PROC_FILENAME)。
 - `mode`: 权限 (0444 - 只读)。
 - `parent`: 父目录 (NULL 表示 /proc 根目录)。

-
- `proc_ops`: `struct proc_ops`, 定义对此 `proc` 文件的操作。
 - `data`: 私有数据指针, 传递给 `proc_ops` 的 `open` 函数, 并最终由 `seq_file` 使用。
 - `proc_remove(entry)`: 移除 `/proc` 文件。
 - `struct proc_ops (buffer_proc_ops)`: `proc` 文件的操作集。
 - `seq_file` API: 用于方便地生成内容可能超过一页的 `/proc` 文件。
 - `single_open(file, show_func, data)`: 辅助函数, 创建一个 `seq_file` 实例, 用单个 `show_func` 生成所有输出。`data` 会存入 `seq_file->private`。
 - `seq_printf(m, fmt, ...)`: 向 `seq_file` 缓冲区 (`m`) 写入格式化数据。
 - `seq_read`, `seq_lseek`, `single_release`: 标准的 `proc_ops` 成员, 配合 `single_open` 使用。
 - `m->private`: 在 `show_func` 中, `seq_file` 实例 `m` 的 `private` 成员即为 `single_open` 时传入的 `data`。
 - 原理:
 1. `buffer_init()` 中用 `proc_create_data()` 创建 `/proc/buffer_status`, 将 `buffer_device` 作为私有数据传入。
 2. 用户 `cat /proc/buffer_status` 时:
 - VFS 调用 `buffer_proc_ops.proc_open` (即 `buffer_proc_open`)。
 - `buffer_proc_open` 使用 `single_open`, 将 `buffer_proc_show` 和 `buffer_device` (作为 `m->private`) 关联到 `seq_file`。
 - VFS 调用 `buffer_proc_ops.proc_read` (即 `seq_read`)。

- seq_read 调用 buffer_proc_show(m, v)。
 - buffer_proc_show 从 m->private 获取 buffer_device，加锁，读取 kfifo 状态和统计信息，用 seq_printf 输出。然后解锁。
 - seq_read 将 seq_file 缓冲区内内容拷贝到用户空间。
3. buffer_exit() 中用 proc_remove() 移除 proc 文件。
- 设计思路:
 - 使用 seq_file API 实现，处理多行或大量数据的输出。
 - single_open 适合这种一次性生成所有内容的场景。
 - buffer_proc_show 负责格式化并输出所有状态信息。
 - 访问共享数据（kfifo 状态, bytes_read/written）时，必须加锁。

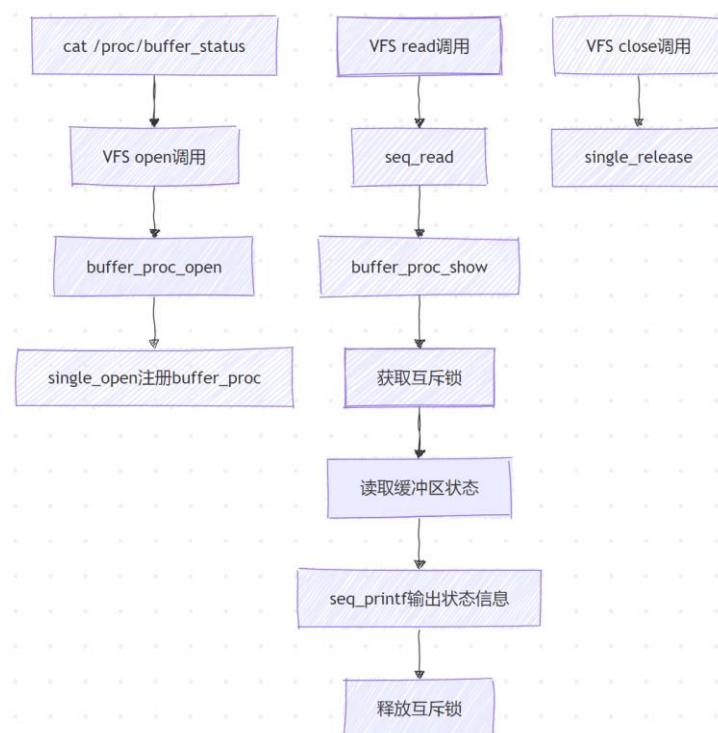


图 4 Proc 文件接口使用流程

1.6 操作流程

1. 加载驱动模块:

- 使用 `sudo insmod buffer_driver.ko` 命令加载内核模块。
- `buffer_init()` 函数会被调用，完成初始化工作。

2. 用户空间操作 `/dev/buffer_dev`:

- 打开设备
 - 用户程序调用 `open("/dev/buffer_dev", O_RDWR | [O_NONBLOCK])`。
 - 内核通过 VFS 调用驱动的 `buffer_open()` 函数。该函数会将 `buffer_device` 结构体指针存入 `file->private_data`，方便后续操作。
- 写入数据:
 - 用户程序调用 `write(fd, data, size)`。
 - 内核调用 `buffer_write()`:
- 读取数据:
 - 用户程序调用 `read(fd, buffer, size)`。
 - 内核调用 `buffer_read()`: 获取互斥锁 `dev->lock`。
- 3. 查看设备状态
 - a. 用户执行 `cat /proc/buffer_status`
 - b. VFS 调用驱动的 `buffer_proc_open()` 和 `seq_read()`
 - c. `buffer_proc_show()` 获取并显示设备状态，包括缓冲区容量、可用数据量、空闲空间、总读写字节数和缓冲区状态
- 4. 卸载驱动程序
 - a. 用户执行 `rmmod buffer_driver`

b. `buffer_exit()`函数被调用，完成资源释放

2. 测试程序编写

2.1 整体分析与设计

本实验共编写了四个相应的测试文件，覆盖了字符设备驱动在阻塞、非阻塞模式下的读写测试，以及设备状态的监控。

2.2 `buffer_writer.c` / `buffer_reader.c` - 缓冲区写入/读取测试

此程序旨在测试 `/dev/buffer_dev` 字符设备的阻塞写入/读取功能。它会持续向设备写入/读取数据块，并观察其行为，特别是当设备驱动程序的内部缓冲区可能已满/空时的情况。每次操作后，它还会通过读取 `/proc/buffer_status` 来显示缓冲区的状态。设计特点如下：

- 参数化测试: 用户可以通过命令行参数自定义每次写入的数据大小和写入操作之间的延时，增加了测试的灵活性。
- 循环写入/读取: 每次写入的数据内容会变化 ('AAAA', 'BBBB', ...), 便于调试时区分不同的数据块。
- 即时状态反馈: 每次写入操作后立即调用 `show_buffer_status()`, 可以清晰地看到缓冲区状态的变化, 验证驱动程序的行为是否符合预期。
- 时间戳记录: `print_timestamp()` 为每条重要的日志信息添加时间戳, 便于分析日志和问题定位, 尤其是在与其他测试程序 (如读取程序) 同时运行时。

2.3 buffer_nonblock.c - 缓冲区非阻塞读写测试

此程序旨在测试 `/dev/buffer_dev` 字符设备的非阻塞读取和写入功能。它会以非阻塞模式打开设备，并尝试进行读写操作。关键在于观察当操作不能立即完成时（从空缓冲区读取或向满缓冲区写入），系统调用是否如预期那样立即返回错误。设计思路如下：

- 使用 `O_NONBLOCK`：通过在 `open()` 系统调用时指定 `O_NONBLOCK` 标志，使得后续的 `read()` 和 `write()` 操作变为非阻塞

```
fd_read = open(DEVICE_PATH, O_RDONLY | O_NONBLOCK);  
fd_write = open(DEVICE_PATH, O_WRONLY | O_NONBLOCK);
```

- 状态监控：使用 `show_buffer_status()` 来观察非阻塞操作对缓冲区的影响（或无影响，如果操作因 `EAGAIN` 失败）。

2.4 buffer_monitor.c - 缓冲区与进程状态监控程序

此程序作为一个独立的监控工具，用于持续地从 `/proc/buffer_status` 读取并显示字符设备缓冲区的状态。同时我们也可以选择通过执行 `ps` 命令来监控一个或多个指定进程的状态（例如，`buffer_writer` 或 `buffer_reader` 进程是否因阻塞而处于睡眠状态）。设计特点如下

- 组合监控：同时监控设备驱动的内部状态（`proc` 文件）和相关用户进程的外部状态（`ps` 命令），提供了更全面的系统行为视图。
- 用户可配置：监控间隔和目标 `PID` 列表均可通过命令行参数指定。
- 清晰的输出：利用清屏和格式化输出，使得监控信息易于阅读。

4 实验程序的难点或核心技术分析

1. 核心思路概述

本实验设计实现了一个带有固定大小循环缓冲区的字符设备驱动程序。该驱动程序支持阻塞/非阻塞读写操作，并通过 `proc` 文件系统提供设备状态的查看接口。核心设计理念包括：

模块化设计：将驱动功能划分为初始化、文件操作、`proc` 文件操作和清理等模块。

阻塞/非阻塞模式：支持两种 I/O 模式，使用等待队列实现阻塞操作。

同步机制：使用互斥锁保护共享资源（缓冲区）。

监控接口：通过 `proc` 文件系统提供实时状态监控。

全面测试：设计专用测试程序验证功能正确性。

2. 关键机制

2.1 `kfifo` 循环缓冲区

在 Linux 内核中，我们使用 `kfifo` API 来定义和管理 FIFO 缓冲区，使用互斥锁确保对临界区代码的安全访问，并使用等待队列实现进程的条件睡眠和唤醒。

```
struct buffer_dev {
    struct kfifo fifo;           // 循环缓冲区
    struct mutex lock;           // 用于同步访问
    wait_queue_head_t read_queue; // 读等待队列
    wait_queue_head_t write_queue; // 写等待队列
    struct cdev cdev;            // 字符设备结构
    unsigned int bytes_read;      // 统计已读数据量
    unsigned int bytes_written;   // 统计已写数据量
};
```

2.2 阻塞读写实现

我们使用等待队列实现进程的条件睡眠和唤醒使用等待队列(wait queue)实现进程睡眠与唤醒机制。

```
while (kfifo_is_empty(&dev->fifo)) {
    mutex_unlock(&dev->lock);
    // 非阻塞模式直接返回错误
    if (filp->f_flags & O_NONBLOCK)
        return -EAGAIN;
    // 阻塞模式下使进程睡眠等待
    if (wait_event_interruptible(dev->read_queue, !kfifo_is_empty(&dev->fifo)))
        return -ERESTARTSYS;

    // 被唤醒后重新获取锁
    if (mutex_lock_interruptible(&dev->lock))
        return -ERESTARTSYS;
}
```

2.3 测试程序设计

测试程序通过系统调用与设备驱动程序交互，实现对 FIFO 缓冲区的读写操作。我们需要处理阻塞和非阻塞操作，以确保测试程序的正确行为。

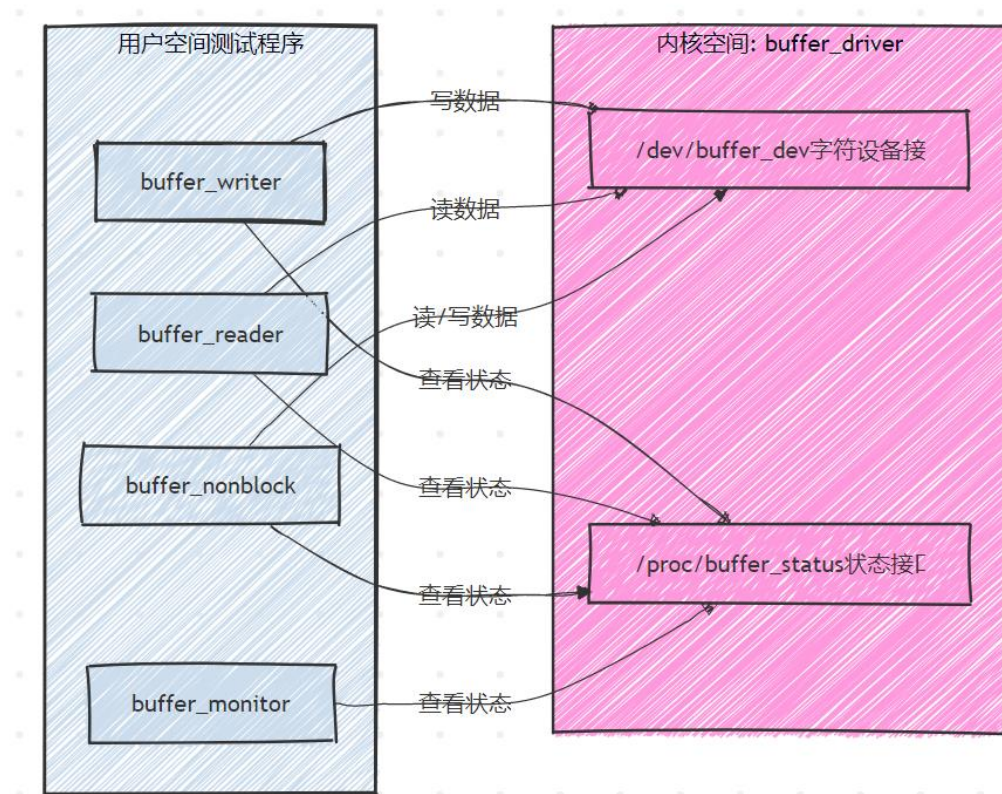


图 5 驱动程序与测试程序的交互

5 开发和运行环境的配置

1. 开发环境

操作系统版本: Ubuntu 22.04.5 LTS
内核版本: 6.6.75.1-microsoft-standard-WSL2
架构: x86_64
编辑器: Visual Studio Code
编译工具: Makefile

2. Makefile 编写

```
MODULE_NAME = buffer_driver
obj-m := $(MODULE_NAME).o

KERNEL_DIR ?= /lib/modules/$(shell uname -r)/build

CC = gcc
CFLAGS = -Wall -g
TEST_PROGS = buffer_writer buffer_reader buffer_monitor buffer_nonblock

all: modules tests

modules:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules

clean-modules:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean

tests: $(TEST_PROGS)

buffer_writer: tests/buffer_writer.c
    $(CC) $(CFLAGS) -o $@ $<

buffer_reader: tests/buffer_reader.c
    $(CC) $(CFLAGS) -o $@ $<

buffer_monitor: tests/buffer_monitor.c
    $(CC) $(CFLAGS) -o $@ $<

buffer_nonblock: tests/buffer_nonblock.c
    $(CC) $(CFLAGS) -o $@ $<

clean-tests:
    rm -f $(TEST_PROGS)

clean: clean-modules clean-tests

install:
    sudo insmod $(MODULE_NAME).ko

uninstall:
    sudo rmmod $(MODULE_NAME)

setup: install
    sudo chmod 666 /dev/buffer_dev

.PHONY: all modules tests clean clean-modules clean-tests install uninstall setup test
```

图 6 Makefile 文件展示

我们可以使用 `make` 命令编译内核模块和测试程序、加载驱动程序，并在多个终端窗口中运行不同的测试程序，既可以通过 `cat` 命令查看 `proc` 文件也可以运行 `buffer_monitor` 程序观察运行结果。

```
# 编译内核模块和所有测试程序

make all

# 加载内核模块

make install          # 或  sudo insmod buffer_driver.ko

# 设置设备文件权限

make setup

# 卸载模块

make uninstall        # 或  sudo rmmod buffer_driver

# 清理所有生成的文件

make clean


# 运行测试程序

./buffer_monitor [interval_ms]

./buffer_reader [read_size] [delay_ms]

./buffer_writer [write_size] [delay_ms]

./buffer_nonblock [mode] [op_size] [delay_ms]
```

6 运行和测试过程

1. 编译和加载内核模块和测试程序

```
knoon@shdn008:~$ cd code/OS_exp/buffer_driver
knoon@shdn008:~/code/OS_exp/buffer_driver$ make all
make -C /lib/modules/6.6.75.1-microsoft-standard-WSL2/build M=/home/knoon/code/OS_exp/buffer_driver modules
make[1]: Entering directory '/home/knoon/kernel_build/WSL2-Linux-Kernel-linux-msft-wsl-6.6.75.1'
  CC [M]  /home/knoon/code/OS_exp/buffer_driver/buffer_driver.o
  MODPOST /home/knoon/code/OS_exp/buffer_driver/Module.symvers
  CC [M]  /home/knoon/code/OS_exp/buffer_driver/buffer_driver.mod.o
  LD [M]  /home/knoon/code/OS_exp/buffer_driver/buffer_driver.ko
  BTF [M]  /home/knoon/code/OS_exp/buffer_driver/buffer_driver.ko
make[1]: Leaving directory '/home/knoon/kernel_build/WSL2-Linux-Kernel-linux-msft-wsl-6.6.75.1'
gcc -Wall -g -o buffer_writer tests/buffer_writer.c
gcc -Wall -g -o buffer_reader tests/buffer_reader.c
gcc -Wall -g -o buffer_monitor tests/buffer_monitor.c
tests/buffer_monitor.c: In function 'main':
tests/buffer_monitor.c:123:9: warning: unused variable 'show_ps' [-Wunused-variable]
   123 |         int show_ps = 1;          // 默认显示进程状态
       |         ^~~~~~
gcc -Wall -g -o buffer_nonblock tests/buffer_nonblock.c
knoon@shdn008:~/code/OS_exp/buffer_driver$ sudo make setup
[sudo] password for knoon:
sudo insmod buffer_driver.ko
sudo chmod 666 /dev/buffer_dev
```

图 7 编译与加载驱动程序

2. 运行测试程序

继续使用上述命令在多个终端运行测试程序，我选择使用 `buffer_monitor` 来观察缓冲区变化与读/写进程的阻塞/被唤醒的同步情况，更详细的情况可见视频内容。从运行结果中可以观察到终端首先打印程序已启动的信息，包括其进程 ID (PID)，以及配置的每次写入大小和写入间隔。接着，程序会调用 `show_buffer_status()` 函数，通过读取 `/proc/buffer_status` 来显示内核驱动中缓冲区的初始状态。

```

knoon@shdn008:~/code/OS_exp/buffer_driver$ ./buffer_writer 5 2000
Buffer Writer started (PID: 22602)
Write size: 5 bytes, Delay: 2000 ms
Press Ctrl+C to exit

----- BUFFER STATUS -----
Buffer Device Status:
-----
Buffer capacity: 32 bytes
Data available: 0 bytes
Free space: 32 bytes
Total bytes read: 0
Total bytes written: 0
Status: EMPTY

```

图 8 阻塞写

```

knoon@shdn008:~/code/OS_exp/buffer_driver$ ./buffer_reader 8 2000
Buffer Reader started (PID: 22905)
Read size: 8 bytes, Delay: 2000 ms
Press Ctrl+C to exit

----- BUFFER STATUS -----
Buffer Device Status:
-----
Buffer capacity: 32 bytes
Data available: 32 bytes
Free space: 0 bytes
Total bytes read: 0
Total bytes written: 32
Status: FULL

```

图 9 阻塞读

```

[19:43:48] Non-blocking READ attempt (5 bytes)...read 5 bytes: 'ZZZZA'

----- BUFFER STATUS -----
Buffer Device Status:
-----
Buffer capacity: 32 bytes
Data available: 27 bytes
Free space: 5 bytes
Total bytes read: 105
Total bytes written: 132
Status: PARTIAL

```

图 10 非阻塞读写

```
knoon@shdn008:~/code/OS_exp/buffer_driver$ ./buffer_monitor 1000 22602 22905
Buffer Monitor started
Update interval: 1000 ms
Monitoring 2 processes: 22602 22905
Press Ctrl+C to exit
[09:58:57] Buffer Status (Update #1)

Buffer Device Status:
-----
Buffer capacity: 32 bytes
Data available: 4 bytes
Free space: 28 bytes
Total bytes read: 136
Total bytes written: 140
Status: PARTIAL
```

图 11 监控程序

3. 卸载驱动程序并清理

```
knoon@shdn008:~/code/OS_exp/buffer_driver$ make uninstall
sudo rmmod buffer_driver
knoon@shdn008:~/code/OS_exp/buffer_driver$ make clean
make -C /lib/modules/6.6.75.1-microsoft-standard-WSL2/build M=/home/knoon/code/OS_exp/buffer_driver clean
make[1]: Entering directory '/home/knoon/kernel_build/WSL2-Linux-Kernel-linux-msft-wsl-6.6.75.1'
CLEAN /home/knoon/code/OS_exp/buffer_driver/Module.symvers
make[1]: Leaving directory '/home/knoon/kernel_build/WSL2-Linux-Kernel-linux-msft-wsl-6.6.75.1'
rm -f buffer_writer buffer_reader buffer_monitor buffer_nonblock
```

图 12 监控程序

7 实验心得和建议

通过本次实验，我对 Linux 内核模块编程、用户空间与内核空间交互、并发控制以及系统调试有了更深刻的理解。在过程中我也遇到了各种 BUG，以下简单的列举一下我发现的问题以及出错的原因：

(1) 字符串处理错误

由于实现的是字符设备驱动程序，为了方便处理字符串和打印输出，确保有

一个位置放置\0 结束符，因此我选择在阻塞模式下使用 `size - 1`。

(2) 内核模块加载错误

在运行 `make` 命令时发现报错 `make: *** [Makefile:52: uninstall] Error 1`，检查 `Makefile` 发现是自己在编写 `make` 命令时有所重复，因此不再需要连续使用 `make install` 和 `make setup`，直接运行 `make setup` 即可

(3) 设备权限错误

在创建 `/dev/buffer_dev` 后发现由于权限不足导致无法访问，因此在 `Makefile` 编写时注意使用 `sudo` 命令。

(4) 阻塞唤醒逻辑错误

忘记在数据写入后唤醒等待读取的进程 (`wake_up_interruptible(&dev->read_queue)`)。

(5) 代码逻辑错误

在持有锁的情况下调用了可能导致睡眠的函数 `wait_event_interruptible` 导致死锁。因此在 `wait_event_interruptible` 前使用 `mutex_unlock` 并在之后 `mutex_unlock_interruptible`。

(6) Makefile 语法错误

由于没怎么接触过 `Makefile` 导致语法问题较多，例如规则的命令行开头为使用 `Tab`。

8 学习和编程实现参考网址

(1) 《操作系统原理》教材

(2) 《自己动手写一个操作系统》

-
- (3) Gemini AI Studio <https://aistudio.google.com/>
 - (4) Linux 官方内核文档 <https://www.kernel.org/doc/html/latest/>